Unnamed_Unit

Unit 2 - Logical operators

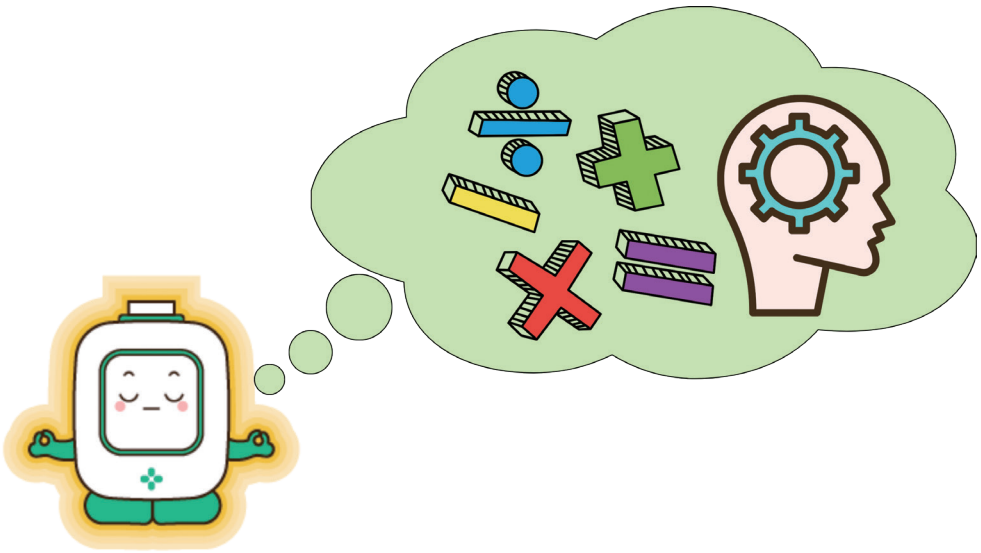
PDF Documents

PDF Document 1: 1734182010947-Chapter 3 - unit 2 - Logical operators.pdf

This PDF document is included in the unit materials.

UNIT 2 LOGICAL OPERATORS

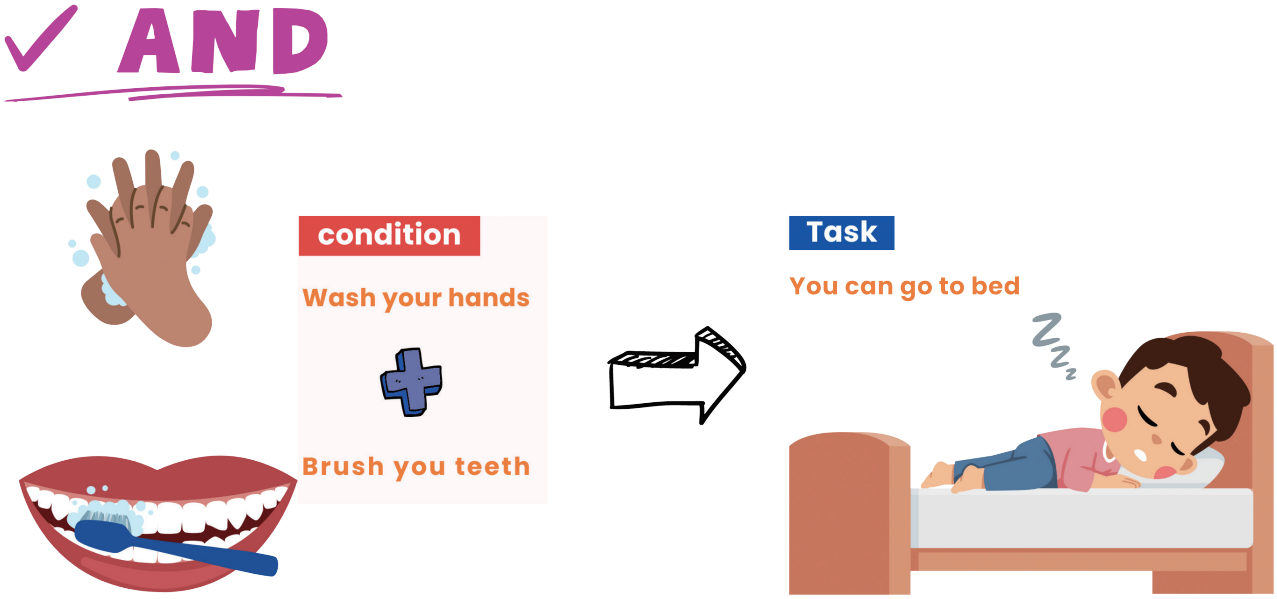
In programming, logical operators are like the brain's decision-making tools, helping computers figure out what to do next. They use simple rules like "AND," "OR," and "NOT" to combine different checks and see if something is true or not.



By learning how to use these operators, you can make your computer programs do more interesting things, like deciding if a game character should move or stay put based on the rules you set.

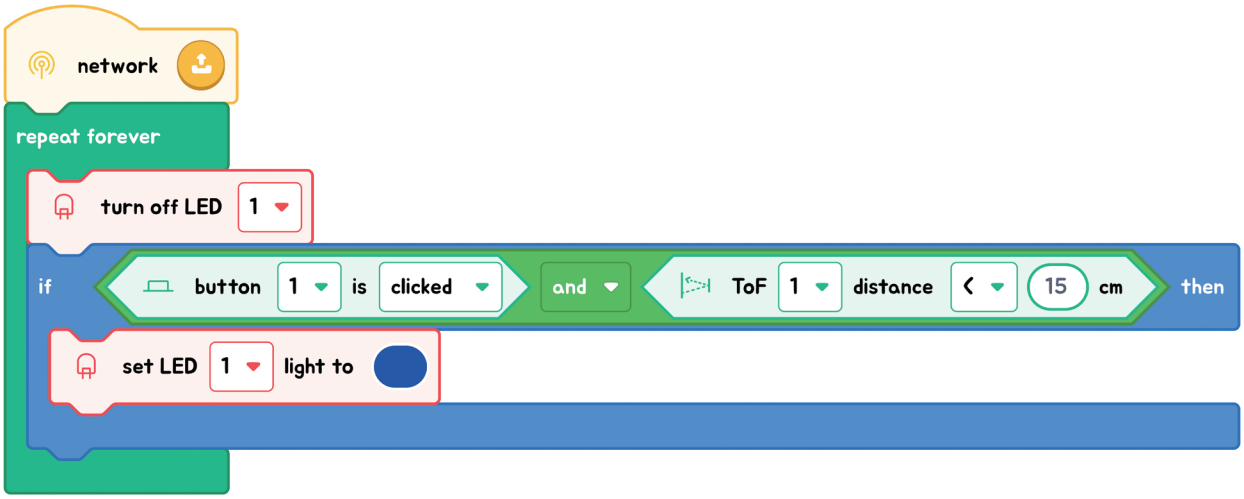
AND

Think of "and" like a bedtime rule. If you wash your hands and brush your teeth, then you can go to bed. If one of the conditions (washing hands or brushing teeth) is not done, you can't go to bed.



In programming, "and" works like a double-check. Both conditions (washing hands or brushing teeth) must be true for the desired outcome (going to bed).

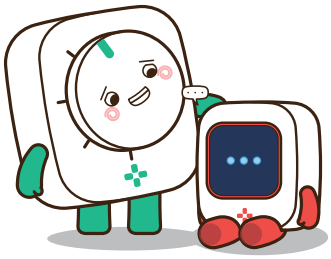
EXPLANATION WITH CODE EXAMPLE



In our example, we're starting with the LED light turned off. Now, we have a special rule for when to turn it on and make it blue. This rule checks two things with "If" statements:

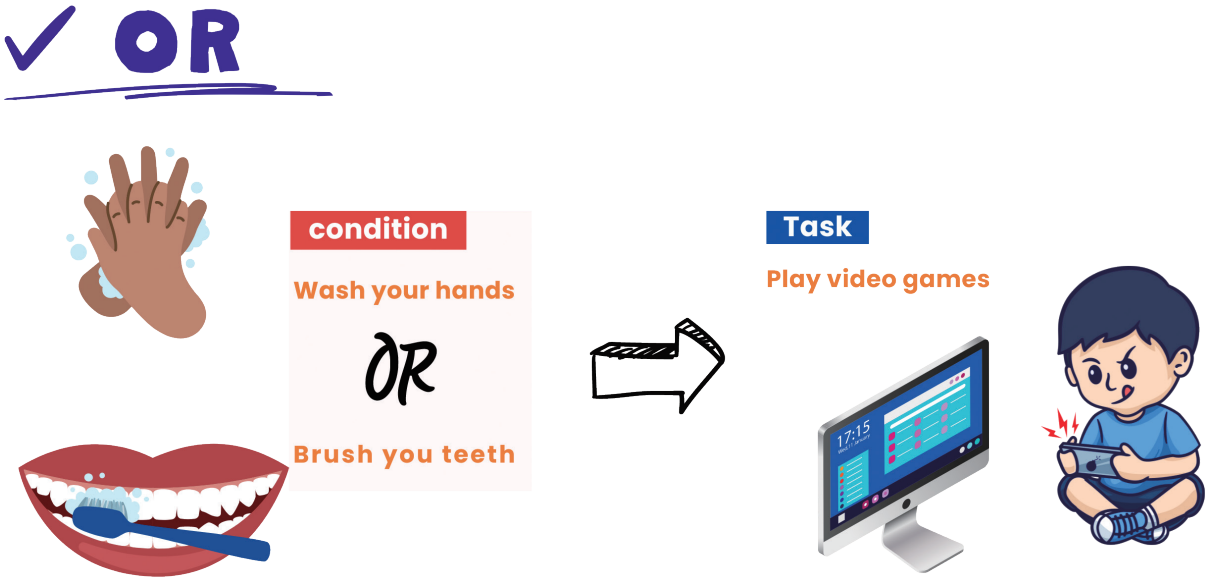
Code Insight

1. **Condition Checking:** The program begins by checking two specific conditions simultaneously: whether a button has been clicked and whether an object is closer than 15cm to the ToF sensor. The combined condition in the code is "If the button is clicked **AND** the ToF's distance is less than 15cm."
2. **Execution if True:** If both conditions are met — the button is clicked and the object is within the specified distance — the program executes the instructions within the "if" statement. In this case, the action taken is "Set the LED light to blue."
3. **Execution if False:** If either or both of the conditions are not met, the default action is maintained, which is to "keep the LED turned off." There's no explicit "else" statement here, but the understanding is that if the conditions for turning the LED blue are not satisfied, the LED remains in its default state (off).



OR

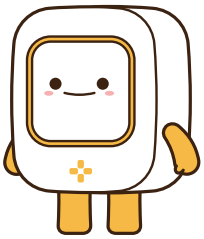
"Or" is like having choices. If you're deciding on a snack and say, "I can have an apple OR a banana," it means you're okay with either one, and you might even choose both if you're extra hungry!



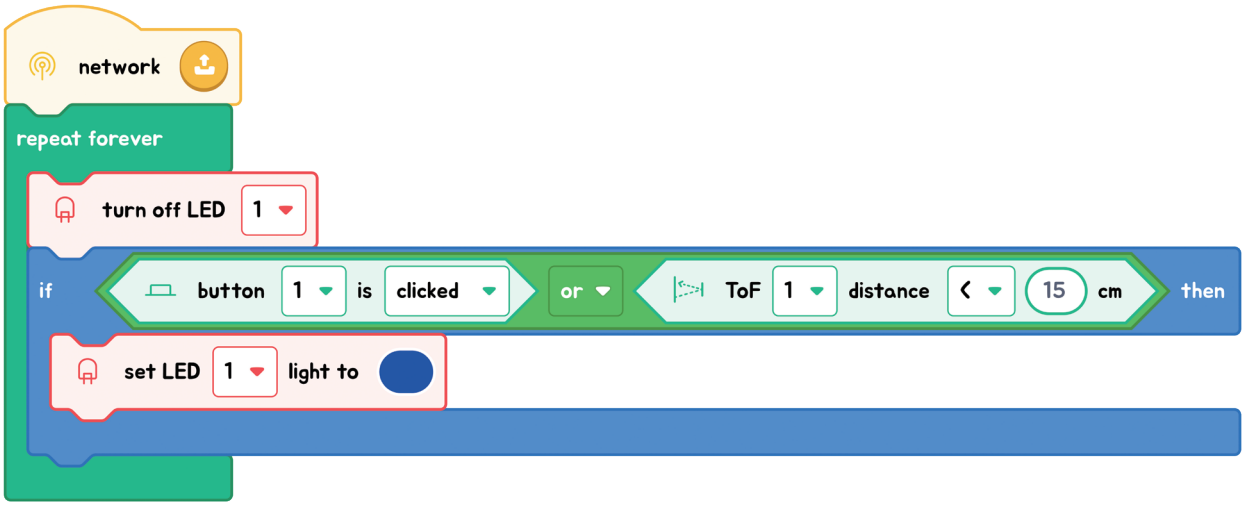
In programming, "and" means both things need to be true, but "or" gives you options—only one thing needs to be true. This helps programmers decide what their code should do in different situations.

TIPS

With "or," it's key to think about how it makes decisions in your code. If any part of an "or" check is true, the whole check is considered true.



EXPLANATION WITH CODE EXAMPLE



In our example, the LED light starts off, meaning it's not lit up. We have a special rule written in our code that decides when the LED should turn blue. This rule uses "or" to check two things:

1) **Is the button clicked?**
   This means we're checking to see if someone presses the button. If yes, the rule is satisfied.

2) **Is something closer than 15cm to the ToF sensor?** This asks if anything is really close, less than 15cm away, to our special sensor called ToF, which can measure distances.

Code Insight

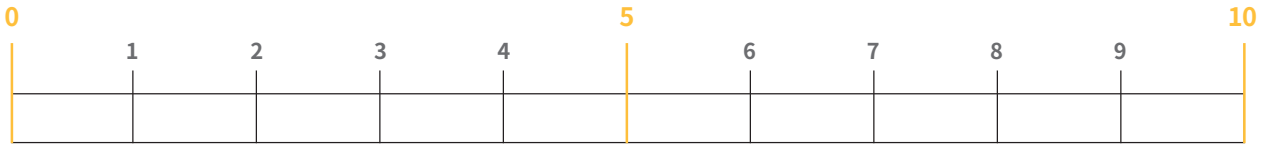
1. **Condition Checking:** Our code first looks at two different situations: whether a button is clicked **or** if something is less than 15cm away from the ToF sensor. It's checking for either of these to be true.
2. **Execution if True:** If at least one of these conditions is true — either the button is clicked or an object is within 15cm of the sensor — then the program activates the instructions within the "if" statement. The action it takes is "Set the LED light to blue."
3. **Execution if False:** If neither condition is met — the button isn't clicked and nothing is close to the sensor — the LED stays in its default state, which is off. There's no need for an "else" statement here because turning off the LED is the default action when neither condition is true.

Exercise

Exercise 1 : "And" in Action

Objective: Understand the "and" operator by coloring numbers that meet specific conditions.

Instructions: You have a number line from 0 to 10. Color all numbers green that are greater than 2 **and** less than 8.

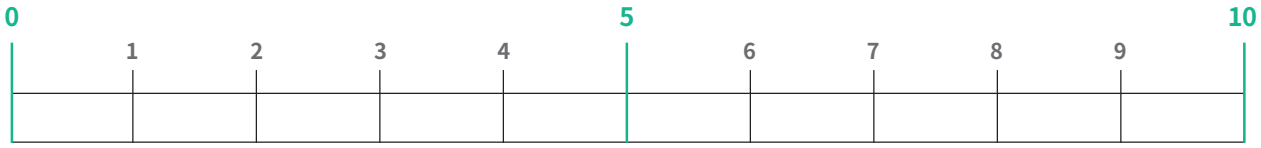


(By coloring these numbers green, you'll highlight the ones that satisfy both conditions, showcasing them as the true values in the code)

Exercise 2 : "Or" Options

Objective: Learn how the "or" operator works by marking two distinct number ranges on your number line, each meeting a different condition.

Instructions: Identify and mark the numbers that are below 4 **or** above 8 on the number line.



exercise 3 : Knowledge check

Which of the following statements best describes the role of the "and" operator?

(A) If one or more conditions are true, the overall condition is true.

(B) The overall condition is true only when all conditions are true.

(C) If one condition is true, the overall condition is true.

(D) The overall condition is true only when all conditions are false.